Koen Essers • Master Internship • April 15, 2026

Notes Week 11

a. New Wind Velocity

I updated the wind velocity in both the *wind angular momentum loss* prescription, as well as in the *wind mass loss* prescription. Both prescriptions now use a constant wind velocity of 15 km s^{-1}.

b. Updated Reference Single Star

Due to the changes to the wind prescription, it is important to generate a new standard single star model that gets used to start the binary simulations from. This new model is different in a couple of ways, which I describe below.

b.1 Updated Wind and History Data

The single star now saves `I_eff`, `rg2`, `M_conv` and `R_conv` to give `evolve.py` everything it needs to accurately compute the orbital evolution. It is now simulated with the updated wind velocity prescriptions and a constant wind velocity of 15 km s^{-1}.

The star only experiences wind mass loss on the EAGB and TPAGB phase, so I *think* it is no problem that the wind velocity is set at 15 km s^{-1} throughout the evolution. The wind angular momentum loss will be 0 during the earlier phases anyway.

b.2 Updated Model saving procedure

The changes in the stellar structure and the orbital evolution are much less drastic during the interpulse region. This makes starting models during this phase easier and more accurate to align to predicted parameters than starting during a thermal pulse.

I devised a new method to save models during the interpulse, but keep track of the largest radius that the star reached during the previous pulse.

```
subroutine save_model_on_radius_increase(id, s, ierr)

    integer, intent(in) :: id
    type(star_info), pointer :: s
    integer, intent(inout) :: ierr

    character(len=200) :: fname
    real(dp) :: R_now, R_last
    real(dp) :: R_max_since_last_save
    real(dp) :: R_max
    integer :: evol_phase
    logical :: needs_save

    needs_save = s% lxtra(lx_needs_save)
    evol_phase = s%x_integer_ctrl(1)

    ! the subroutine uses the current stellar radius
    ! and the last saved stellar radius to determine
    ! if a model should be saved.
    R_now = s% photosphere_R
    R_last = s% xtra(x_last_saved_R)
    R_max_since_last_save = s%
     ↪  xtra(x_max_R_since_last_save)

    ! if true: save a model

    ! this criterion produces many models,
    ! but using this routine is only required once
    ! for every mass star!
    if (((R_now - R_last > 10d0) .or. (R_now > 1.25d0 *
     ↪  R_last)) .and. (evol_phase > 1) .and. (evol_phase
     ↪  /= 5)) then
        write(fname, fmt="(a7,f0.3,a7,i0,a4)") 'models/',
         ↪  R_now, 'Rsun_TP', s% ixtra(ix_TP_count),
         ↪  '.mod'

        call star_write_model(id, fname, ierr)

        s% need_to_update_history_now = .true.
        s% need_to_save_profiles_now = .true.

        open(1, file="last_saved_R.dat",
         ↪  status="replace")
        write(1,*) R_now
        close(1)

        if (ierr /= 0) then
            write(*,*) 'error writing model'
            ierr = 0
        end if

        s% xtra(x_last_saved_R) = R_now
        s% xtra(x_max_R_since_last_save) = R_now
    end if

    if ((R_now - R_max_since_last_save > 0d0) .and.
     ↪  (evol_phase == 5) .and. (s% power_h_burn / s%
     ↪  power_nuc_burn < 0.7)) then
        s% xtra(x_max_R_since_last_save) = R_now
        s% lxtra(lx_needs_save) = .true.
    end if

    if ((s% power_h_burn / s% power_nuc_burn > 0.7) .and.
     ↪  (s% lxtra(lx_needs_save)) .and. (evol_phase ==
     ↪  5)) then

        write(fname, fmt="(a7,f0.3,a7,i0,a4)") 'models/',
         ↪  R_max_since_last_save, 'Rsun_TP', s%
         ↪  ixtra(ix_TP_count), '.mod'

        call star_write_model(id, fname, ierr)

        s% need_to_update_history_now = .true.
        s% need_to_save_profiles_now = .true.

        open(1, file="last_saved_R.dat",
         ↪  status="replace")
        write(1,*) R_now
        close(1)

        if (ierr /= 0) then
            write(*,*) 'error writing model'
            ierr = 0
        end if

        s% xtra(x_last_saved_R) = R_max_since_last_save
        s% lxtra(lx_needs_save) = .false.
    end if


end subroutine save_model_on_radius_increase
```

I think this is quite a big improvement compared to the last method. Now, we always get the *maximum* radius during a thermal pulse, so we can more accurately infer the maximum ratio of $R_{\text{star}}/R_{\text{RL}}$ during the thermal pulses and select a better starting model.[1]

1 However, see section g.

> **NOTE:** This new method will *not* work as expected for heavier stars! As we have seen, heavier stars can have radii that are larger during the interpulse than the previous thermal pulse. An altered method has to be created.

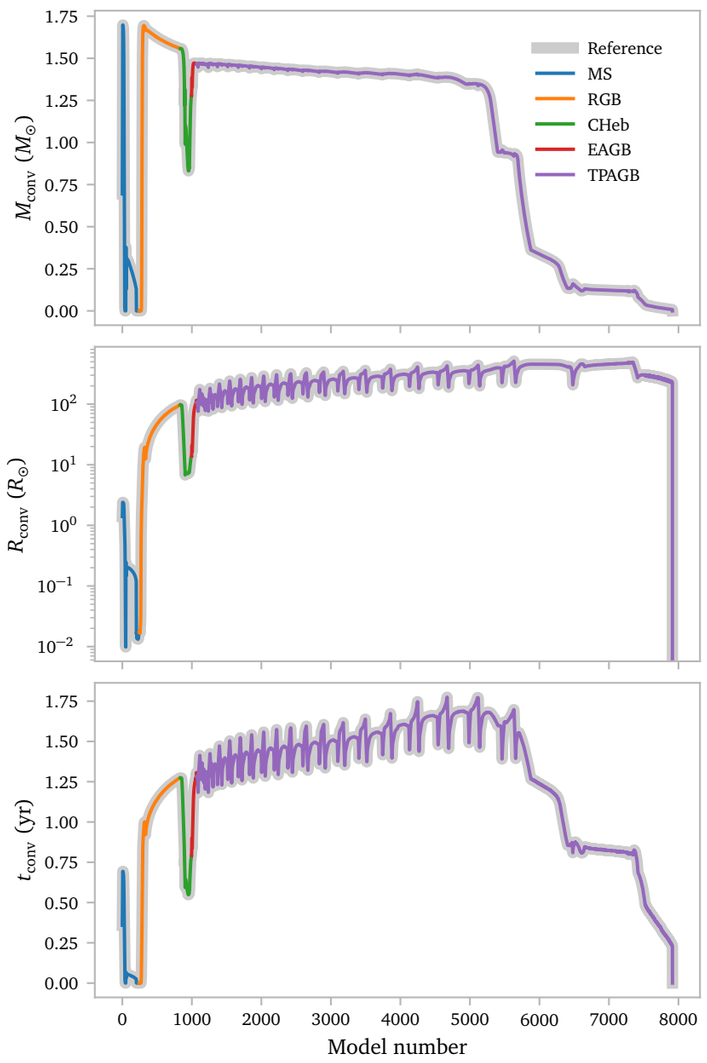
c. Comparing with evolve.py

> **NOTE:** This section still uses the original solar model to evolve the starting binary from. This means that the star *up to the point of the start of the binary evolution* still used the Vassiliadis & Wood (1993) wind velocity, but as we have seen last week, this does *not* alter the mass lost in any meaningful way. This also means the model still starts during a thermal pulse. Later experiments *do* use the new standard star as *input*.

Before, I only loaded the TPAGB data from the Rees models into the Star class from `evolve.py`. To accurately predict starting Roche Lobe radii, q-values and stellar spin parameters, it is important to model the previous evolutionary phases as well. To do this, I modify the `read_stellar_models` function to read the complete standard star data:

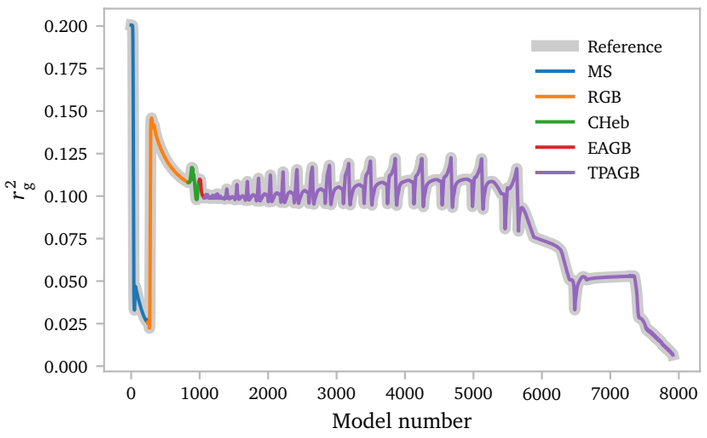
```
def read_stellar_models():
    n = [0]
    for phase in ["MS", "GB", "CHeB", "EAGB", "TPAGB"]:
        data = mr.MesaData(
            f"""/home/koen/master-internship/mesa-models/
            standard-2msun-v2/LOGS/{phase}/history.data"""
        )
        n.append(n[-1] + len(data.model_number[1:]))

    combined_star = {}
    for bulk_name in data.bulk_names:
        combined_star[bulk_name] = np.empty(n[-1])
    combined_star["phase"] = np.empty(n[-1])

    for i, phase in enumerate(["MS", "GB", "CHeB", "EAGB", "TPAGB"]):
        data = mr.MesaData(
            f"""/home/koen/master-internship/mesa-models/
            standard-2msun-v2/LOGS/{phase}/history.data"""
        )
        for bulk_name in data.bulk_names:
            if bulk_name in ["model_number", "star_age"] and i != 0:
                combined_star[bulk_name][n[i] : n[i + 1]] = (
                    data.data(bulk_name)[1:] + combined_star[bulk_name][n[i] - 1]
                )
            else:
                combined_star[bulk_name][n[i] : n[i + 1]] = data.data(bulk_name)[1:]
        combined_star["phase"][n[i] : n[i + 1]] = i * np.ones(n[i + 1] - n[i])

    Stars = [StellarModel(combined_star, n)]
    return Stars
```

Below, I show that this ensures the `Star` class from `evolve.py` has the same `M_conv`, `R_conv`, `t_conv` and `rg2` values as the MESA reference star. It also has access to all other properties required for computing the tides and wind.

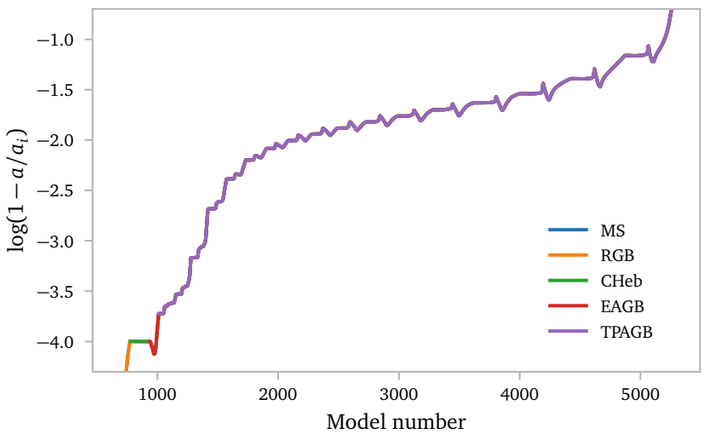


The `evolve.py` values are identical to the history values, is that great!

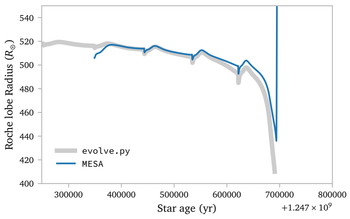
Now for r_g^2



If we use these values to compute the wind and tides, we can see the effects of all phases on the separation and Roche lobe radius for arbitrary starting radii and mass ratios.



Now, we can compare the Roche lobe radius from `evolve.py` with the radius from `MESA`. If everything is correct, it *should* be nearly identical.



Clearly, it is *not* nearly identical. The `evolve.py` model seems to show a much sharper decline in Roche lobe radius. My suspicion was that this is due to the stellar wind prescription, since it does not seem to affect the first and second pulse as much as it affects the third and final pulse.

Below, I show the code that I use that is responsible for the angular momentum loss due to winds:

```
! src/bin/additional_routines.inc
subroutine wind_loss(binary_id, ierr)
    integer, intent(in) :: binary_id
    integer, intent(out) :: ierr
    type(binary_info), pointer :: b
    real(dp) :: q, vw_over_vorb
    real(dp) :: beta, eta
    real(dp) :: omega, I_eff
    real(dp) :: domega_dt_wind

    ierr = 0
    call binary_ptr(binary_id, b, ierr)
    if (ierr /= 0) then
        write (*, *) 'failed in binary_ptr'
        return
    end if

    ! --- compute q == m_accretor / m_donor and v
    q = b%m(b%a_i)/b%m(b%d_i)
    vw_over_vorb = compute_vwind(b%s_donor)/compute_vorbit(b%m(1) + b%m(2), b%separation)
    b%s_donor%xtra(x_vw_over_vorb) = vw_over_vorb

    ! --- get beta and eta from Saladino
    beta = compute_beta_Sal(q, vw_over_vorb)/sqrt(1 - pow2(b%eccentricity))
    b%s_donor%xtra(x_beta) = beta
    eta = compute_eta_Sal(q, vw_over_vorb)
    b%s_donor%xtra(x_eta) = eta

    ! --- star angular momentum loss due to winds
    ! compute the change in spin frequency. this
    ! gets used in extra_binary_finish_step.
    omega = b%s_donor%xtra(x_omega)
    I_eff = b%s_donor%xtra(x_I_eff)
    domega_dt_wind = (b%mdot_system_wind(b%d_i) &
                     *(b%s_donor%photosphere_r*Rsun)**2*omega/1.5)/I_eff
    b%s_donor%xtra(x_domega_dt_wind) = domega_dt_wind

    ! --- mass lost from vicinity of donor
    b%jdot_ml = b%mdot_system_wind(b%d_i) &
                *(1 - beta) &
                *eta &
                *(b%separation)**2 &
                *sqrt(1 - pow2(b%eccentricity)) &
                *2*pi/b%period

end subroutine wind_loss
```

The important flaw here is the use of the variable `b%mdot_system_wind(b%d_i)` in combination with the term `(1-beta)`.

If we look at how `b%mdot_system_wind(b%d_i)` is defined in the MESA source code:

```
b% mdot_system_wind(b% d_i) = b% s_donor% mstar_dot - b% mtransfer_rate &
            + b% mdot_wind_transfer(b% a_i) - b% mdot_wind_transfer(b% d_i)
```

we can see that it uses the total $\dot{M}$ for the donor star, subtracts the mass transfer term due to RLOF `b% mtransfer_rate`, adds the wind mass transfer from the accretor `b% mdot_wind_transfer(b% a_i)`, which is always zero if the accretor is a point mass, and subtracts the wind mass transfer from the donor `b% mdot_wind_transfer(b% d_i)`. Effectively, this means

$$\dot{M}_{\rm wind,sys} = \dot{M}_{\rm d,wind} - \dot{M}_{\rm d,wind\text{-}transfer},$$

and the problem is already becoming clear. Below, I show the definition of `b% mdot_wind_transfer(b% d_i)`:

```
! $MESA_DIR/binary/private/binary_mdot.f90
! solve wind mass transfer
    ! b% mdot_wind_transfer(b% d_i) is a negative number that gives the
    ! amount of mass transferred by unit time from the donor to the
    ! accretor.
    call eval_wind_xfer_fractions(b% binary_id, ierr)
    if (ierr/=0) then
        write(*,*) "Error in eval_wind_xfer_fractions"
        return
    end if
    b% mdot_wind_transfer(b% d_i) = b% s_donor% mstar_dot * &
        b% wind_xfer_fraction(b% d_i)
```

Here, `b% wind_xfer_fraction(b% d_i)` is exactly `beta`, so really,

$$\dot{M}_{\rm wind,sys} = \dot{M}_{\rm d,wind}(1-\beta),$$

and my subroutine above is effectively computing

$$\dot{J} = \dot{M}_{\rm wind}(1-\beta)^2\eta a^2\Omega_{\rm orb}.$$

which is *wrong*! To check whether this truly is the cause of the discrepancy, I decide to modify the `evolve.py` script to also use and extra factor of $1-\beta$ in the $\dot{a}/a$ computation, since this is *much* quicker than simulating the possibly correct result in MESA. To see how the extra factor of $1-\beta$ affects the separation, I derived the following expression using Saladino et al. (2018):

$$\begin{aligned}\left(\frac{\dot{a}}{a}\right)_{\rm wind} &= 2\left(\frac{\dot{J}}{J}\right)_{\rm wind} - 2\frac{\dot{M}_{1,\rm wind}}{M_1} - 2\frac{\dot{M}_{2,\rm wind}}{M_2} + \frac{\dot{M}_{1,\rm wind}-\dot{M}_{2,\rm wind}}{M_1+M_2}\\ &= 2\left(\frac{\dot{J}}{J}\right)_{\rm wind} - 2\frac{\dot{M}_{\rm d,wind}}{M_{\rm d}} + 2\frac{\beta\dot{M}_{\rm d,wind}}{M_{\rm a}} + \frac{\dot{M}_{\rm d,wind}-\beta\dot{M}_{\rm d,wind}}{M_{\rm d}+M_{\rm a}}\end{aligned}$$

From the expression above, and $J=\mu a^2\Omega_{\rm orb}$, we can deduce that

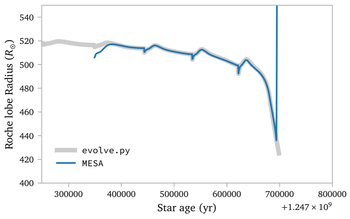
$$\begin{aligned}\frac{\dot{J}}{J} &= \frac{\dot{M}_{\rm d,wind}(1-\beta)^2\eta(M_{\rm d}+M_{\rm a})}{M_{\rm d}M_{\rm a}} \quad \text{and thus}\\ \left(\frac{\dot{a}}{a}\right)_{\rm wind} &= 2\frac{\dot{M}_{\rm d,wind}}{M_{\rm d}}\left[\frac{(1-\beta)^2\eta(M_{\rm d}+M_{\rm a})}{M_{\rm a}} - 1 + \frac{\beta M_{\rm d}}{M_{\rm a}} + \frac{M_{\rm d}(1-\beta)}{2(M_{\rm d}+M_{\rm a})}\right]\end{aligned}$$

With $\tilde{q}=M_{\rm d}/M_{\rm a}$:

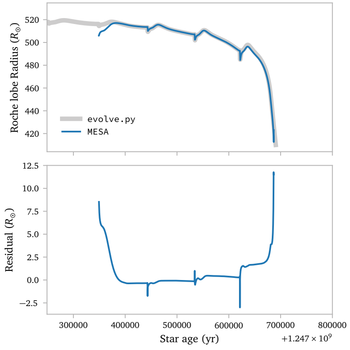
$$\frac{\dot{J}}{J} = 2\frac{\dot{M}_{\rm d,wind}}{M_{\rm d}}\left[(1-\beta)^2\eta(1+\tilde{q}) - 1 + \tilde{q}\beta + \frac{\tilde{q}(1-\beta)}{2(1+\tilde{q})}\right].$$

This expression differs only in the in the $(1-\beta)^2$ term. The correct equation only has $(1-\beta)$.

If we use this expression in `evolve.py`, we see the following:

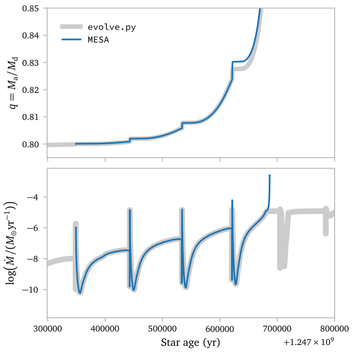


Excellent agreement! This makes me optimistic that the accidental double $(1-\beta)$ factor is the only (significant) discrepancy between the two codes. Next, I fix the mistake and run a new MESA model.



The `evolve.py` and MESA implementation agree very well *but* after the second pulse, the `evolve.py` implementation seems to predict slightly higher Roche lobe radii. This is more apparent in the bottom panel, where I plot the residuals.

I predict this is due to non-negligible amounts of RLOF mass transfer during the thermal pulses. To confirm this, we can plot M for both methods, as well as q for both methods. This can also explain the slight variations in TPAGB interpulse durations.



These plots clearly show that RLOF affects the mass ratio, and thus also the Roche lobe radius.

d. Starting Conditions

Now that the MESA models align well with the `evolve.py` models, at least for the single model that I tested, I want to use this `evolve.py` script to find starting conditions for the MESA models.

This way, it is no longer required to guess the inputs values for `evolve.py` to compare with the MESA models, *and* the `evolve.py` script can actually be used to find ZAMS values for the orbital properties, evolve the star, and find the equivalent starting binary orbital parameters.

d.1 Code

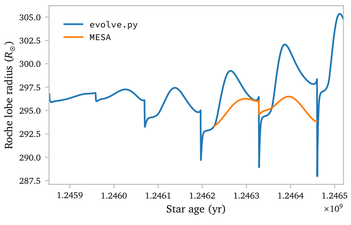
```
models_dict = get_model_dict()
tpagb_age = _get_tpagb_age()
ref_eagb_age = mr.MesaData(f"{ref_dir}/eagb.data").star_age[-1]

Star = read_stellar_models(standard_dir)[0]

for R in Rs:

    model = find_model(R)
    mass = model["M"]
    model_path = f"{grid_dir}/models/{model["name"]}"
    mod_data = mr.MesaData(model_path)
    model_age = _find_initial_age(mod_data) + tpagb_age

    for q in qs:
        a_init = inv_roche_lobe(R, q)
        [Star, Options, q_init, a_init, e_init, Bins] = call_evolution(
            Star, q, a_init, simple_only=True
        )

        bin = Bins[0]
        index = np.argwhere(bin.age > model_age)[0][0] - 1
        index_star = np.argwhere(Star.age > model_age)[0][0] - 1

        m1 = bin.m1[index]
        m2 = bin.m2[index]
        a = bin.a[index]
        omega = bin.spin1[index]

        varcontrol = Star.varcontrol[index_star]

        print(f"\nR = {R:.0f}, q = {q:.2f}")
        run_subdir = f"runs/R{R:05.2f}_q{q:.3f}"
        run_dir = f"{grid_dir}/{run_subdir}"
        print(f"copying reference binary to {run_subdir}...")
        shutil.copytree(f"{grid_dir}/reference-binary", run_dir)
        inlist_bin = f"{run_dir}/inlist_project"
        inlist_star = f"{run_dir}/inlist1"
        print(f"copying {model["name"]} to {run_subdir}...")
        shutil.copyfile(model_path, f"{run_dir}/start.mod")
        print(f"changing inlist...")
        change_inlist_bin(a, m1, m2, inlist_bin)
        change_inlist_star(omega, varcontrol, inlist_star)
```

Below, I show the functions `changea_inlist_bin` and `change_inlist_star`. These are responsible for actually making sure the binaries are evolved with the correct starting values!

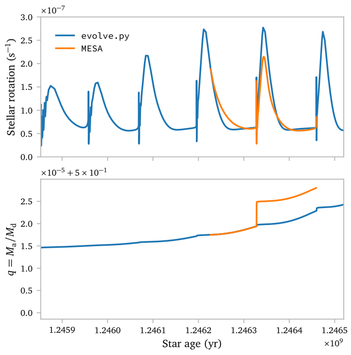
```
def change_inlist_bin(a, m1, m2, inlist_path):

    with open(inlist_path, "r") as f:
        lines = f.readlines()

        new = []
        for line in lines:
            if "m1" in line:
                new.append(f"\tm1 = {m1}d0  ! donor mass in Msun\n")
            elif "m2" in line:
                new.append(f"\tm2 = {m2}d0  ! donor mass in Msun\n")
            elif "initial_separation_in_Rsuns" in line:
                new.append(f"\tinitial_separation_in_Rsuns = {a}d0 ! in Rsun units\n")
            else:
                new.append(line)

    with open(inlist_path, "w") as f:
        f.writelines(new)


def change_inlist_star(omega, varcontrol, inlist_path):

    with open(inlist_path, "r") as f:
        lines = f.readlines()

        new = []
        for line in lines:
            if "x_ctrl(3)" in line:
                val = f"{omega:.16e}".replace("e", "d")
                new.append(f"\tx_ctrl(3) = {val}  ! initial rotation rate\n")
            elif "x_ctrl(4)" in line:
                val = f"{varcontrol:.3e}".replace("e", "d")
                new.append(f"\tx_ctrl(4) = {val}  ! initial varcontrol\n")
            else:
                new.append(line)

    with open(inlist_path, "w") as f:
        f.writelines(new)
```

d.2 First Test

Below, I show the first test. In this case, it is a model starting with an initial Roche lobe radius of $R_{RL} = 300\,R_\odot$ and $q = 0.5$



It is clearly wrong, but the starting Roche lobe radius is correct. This means the separation finding mechanism is working as intended. Below, I check if q and Ω_{star} are also working correctly.

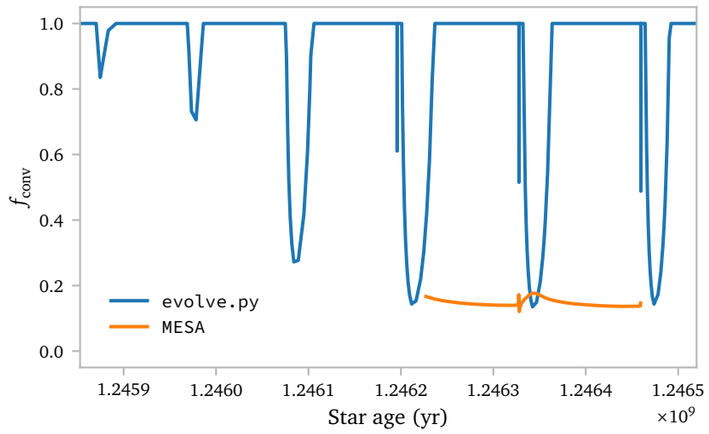


It seems like these are working as intended as well!

d.3 Problem with Tides

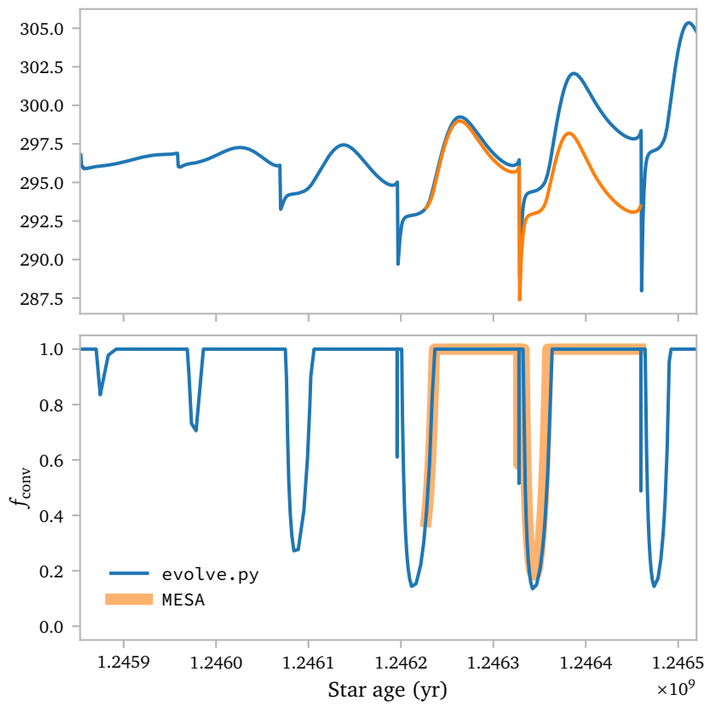
After debugging for *way* too long, I finally traced the issue back to the fast tides factor f_{conv}, which was still using the old (now static) stellar rotation instead of the updating new value.

Below, I show f_{conv} for both the `evolve.py` and the `MESA` model.



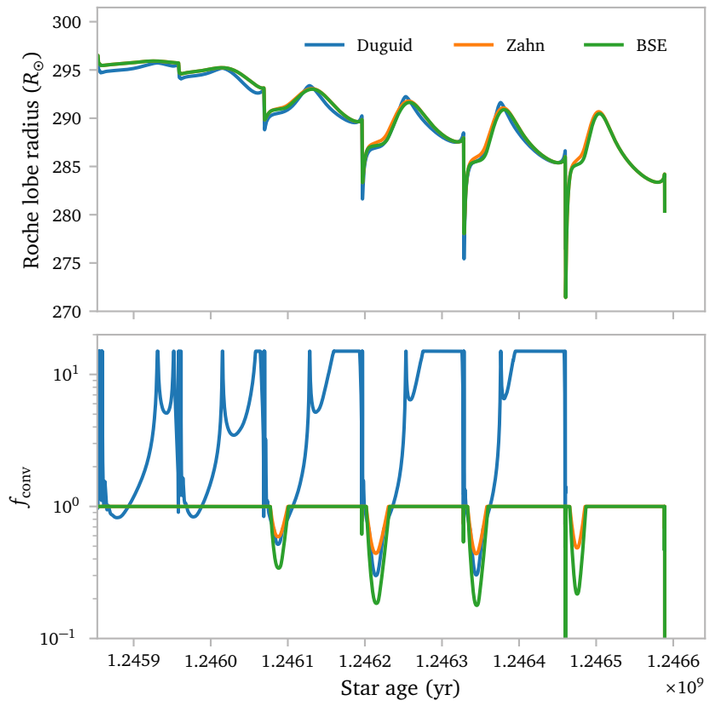
Not only is it important that I noticed this issue, it also shows that f_{conv} really is not 1 throughout the TPAGB evolution. During the third dredge up, it will decrease quite significantly, *and* as the plot above shows, it also has an important effect on the evolution of the tides.

After fixing the mistake, we get the following plot.



During the first interpulse, `MESA` and `evolve.py` agree quite well with eachother. *However*, during and after the first pulse that `MESA` evolves, the models diverge quite drastically. Something is clearly still wrong.

Before going into the remaining problem, I first want to investigate how much the different f_{conv} prescriptions affect the overall tides. The figure below shows the differences for the star with an initial Roche lobe radius of $R_{\text{RL}} = 300\,R_\odot$.

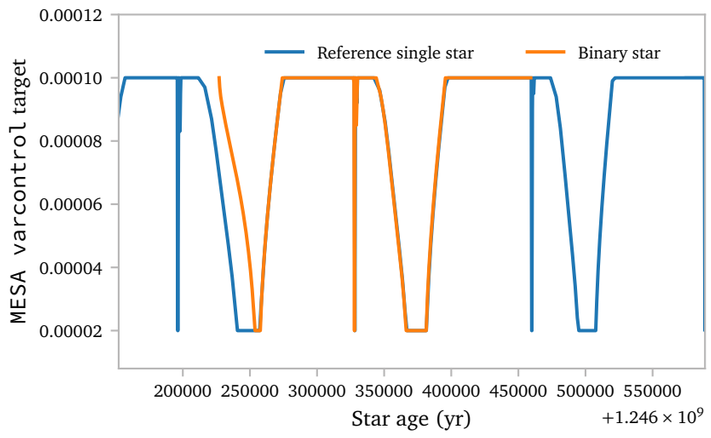


Luckily, for physically motivated prescriptions, the differences are not nearly as significant on the TPAGB. I will continue to use the BSE prescription.

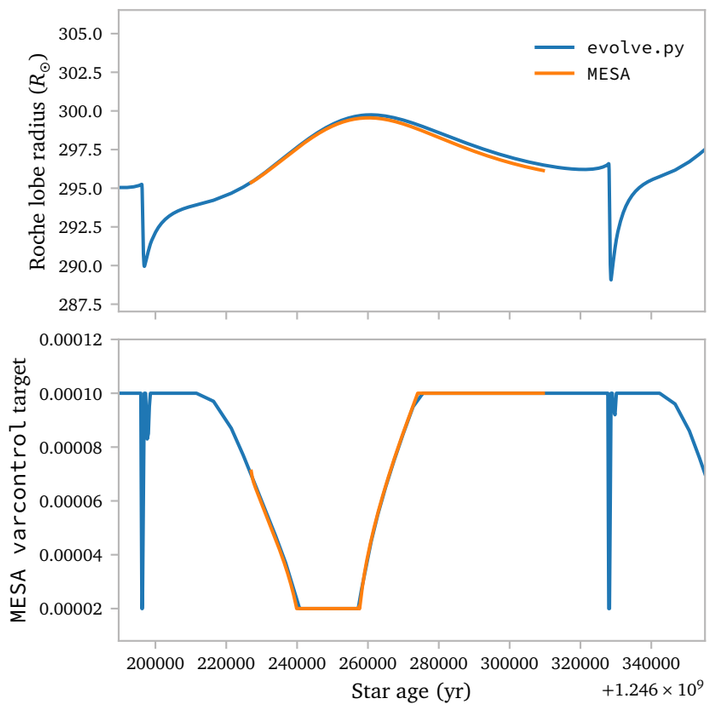
Note: These plots are made with a star model with a better temporal resolution. I will come back to this later. Duguid is pretty unstable when using the lower resolution model.

d.4 Problem with varcontrol

The Rees et al. (2024) `MESA` models use a varying `varcontrol` prescription to completely resolve the third dredge up. Most of my models start in the third dredge up, and thus the reference single star has a lower `varcontrol` value at the time of the model. MESA does not however save this in the `models.mod` files. This results in the following behaviour:



An easy fix for this is to simply set the binary star with the same varcontrol target as the single star when it is initiated.[2] After the fix, we obtain the following



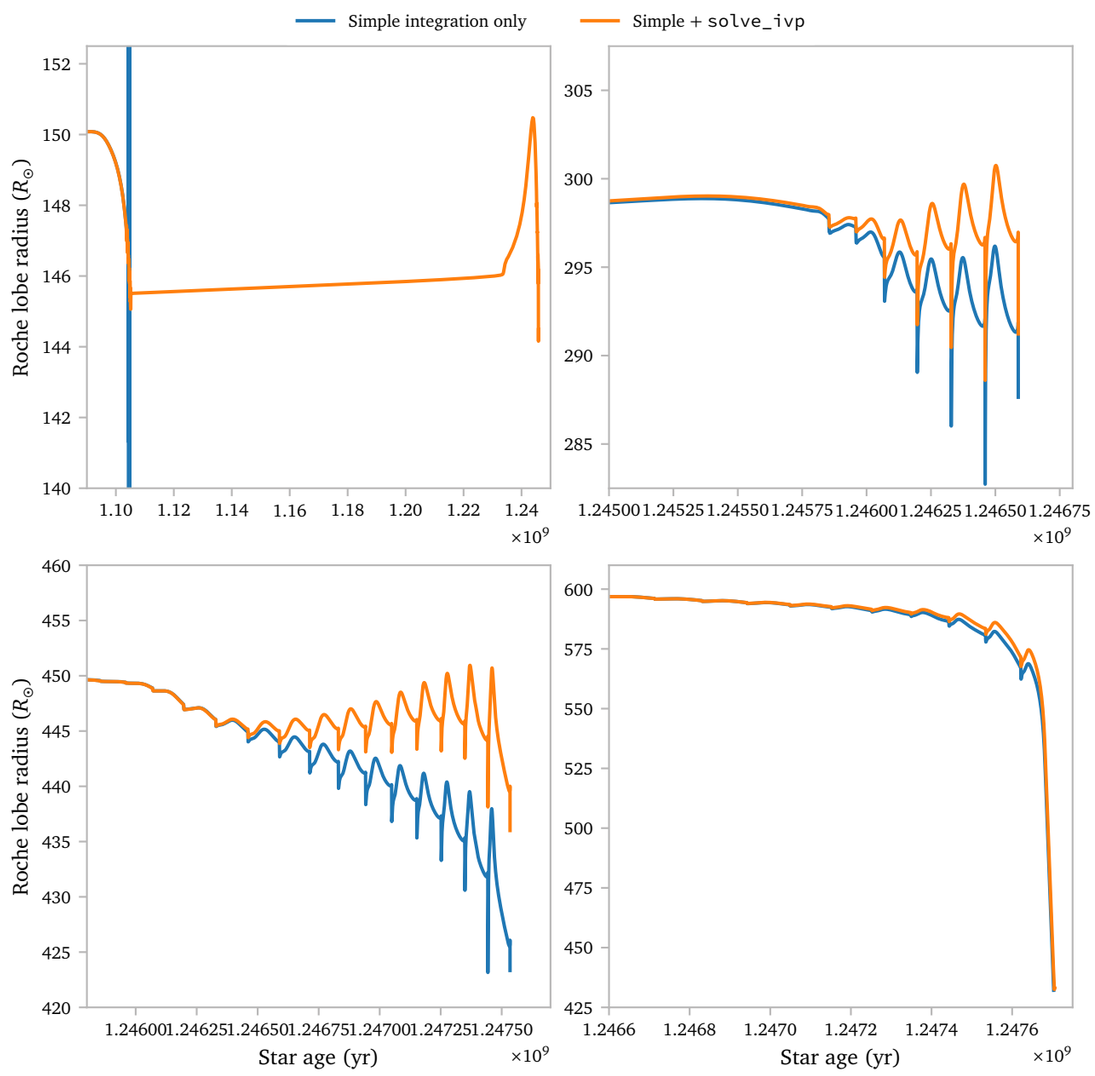
[2] This behaviour is already shown in the grid generator code above.

Seems like it did not significantly affect the tides, but it is still an important change, since this ensures the dredge up is correctly resolved and thus the abundances can more accurately be predicted.

I might have stopped the simulation a bit early, but at this point I was already certain the differences between the `evolve.py` script and the `MESA` code we not caused by the different varcontrol target.

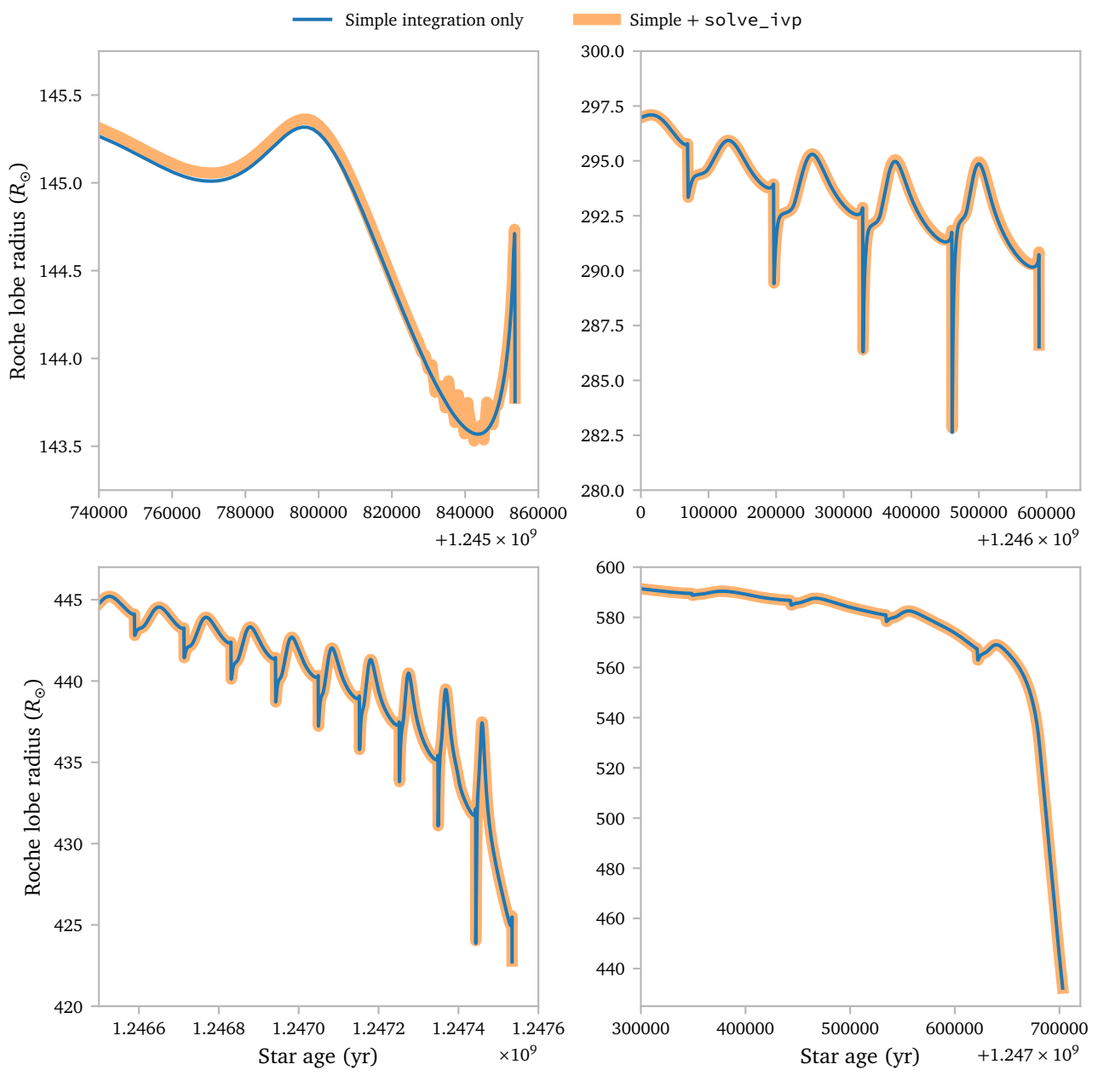
d.5 Simple integration

The actual differences are probably due to the differences in integration scheme between `MESA` and the `evolve.py` script. Below, I plot the output of the `evolve.py` script for two different integration schemes. In one case, I `only` integrate using the simple integration scheme which is just $\Delta x/\Delta t \cdot \Delta t$. The other case uses an ε-parameter to change between the simple integration, and a sophisticated ODE solver from scipy.



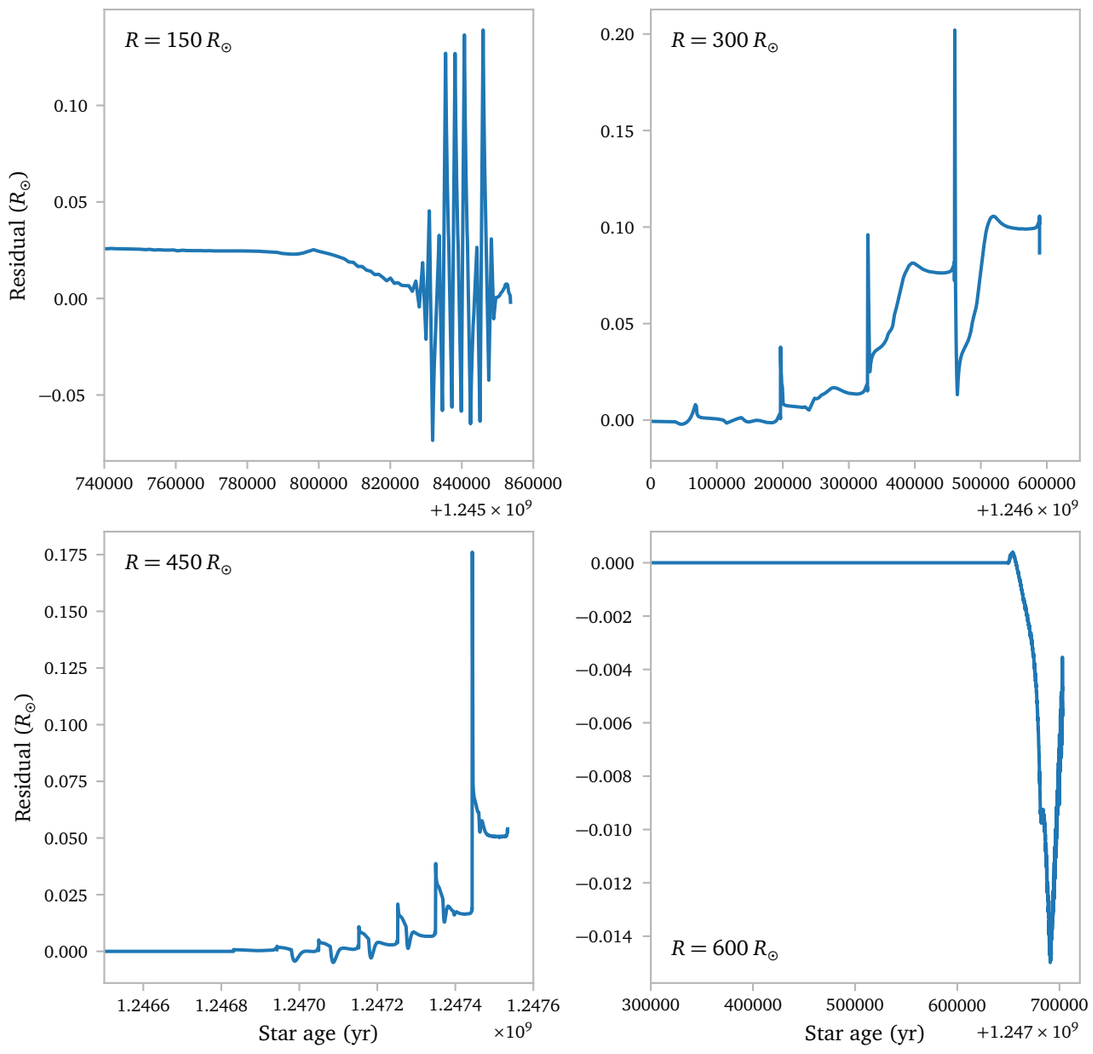
This is quite concerning, since our MESA implementation uses the simple integration scheme. However, the `evolve.py` script uses MESA history files, which in this case, only contain 1/10 of the model steps. We *could* simulate the reference star again, this time saving *every* model step. This results in large file sizes and considerably slows down the `evolve.py` code. However, I think it is an important test.

I simulate another single star, this time saving all model steps and input this into the `evolve.py` script. The results are shown below:

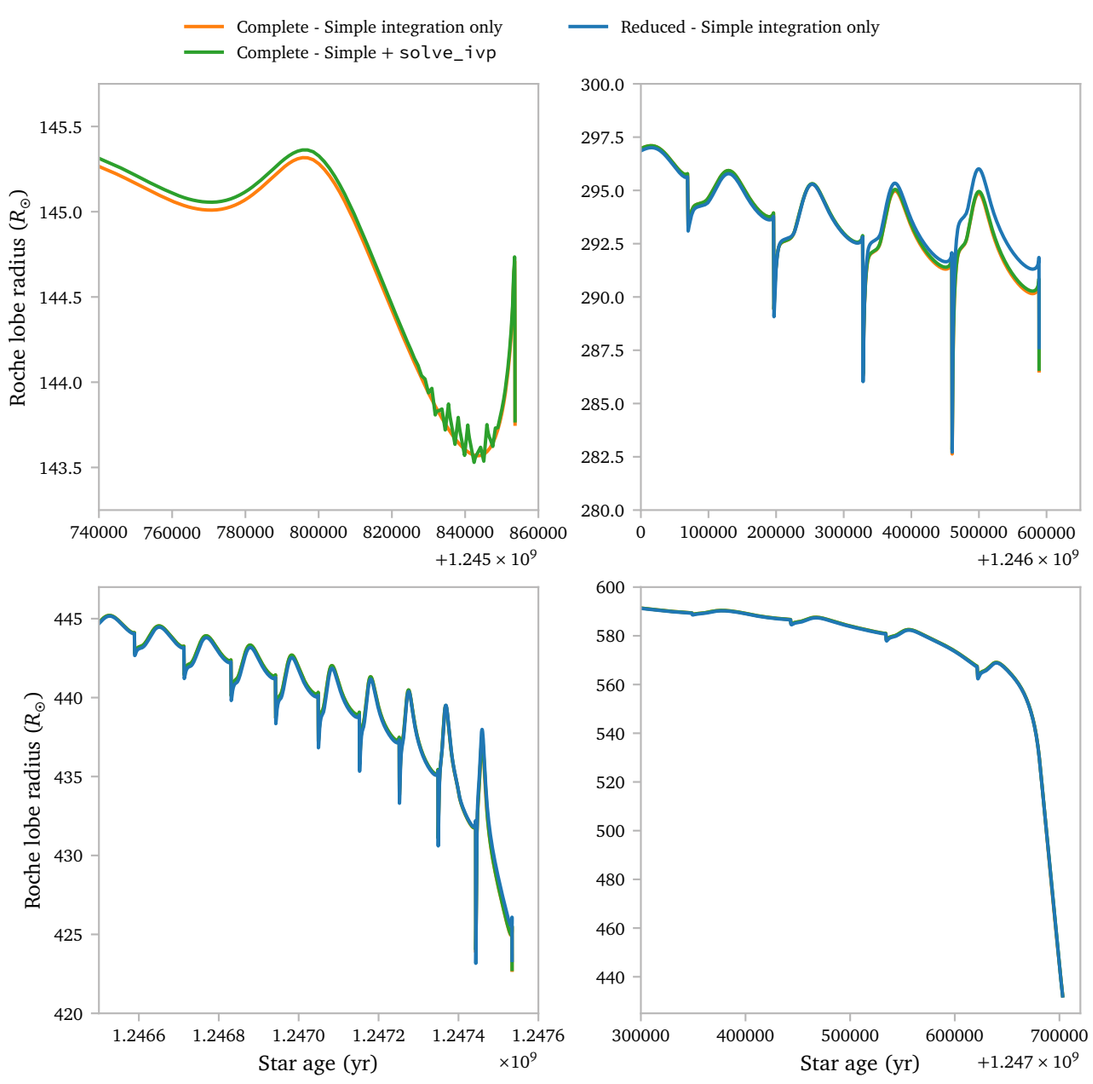


This time, luckily for us, the simple and direct integration agree much better. They are, however, still not completely in agreement. In order to resolve this, the MESA binary models should take smaller timesteps, or a sophisticated ODE solver should be used inside MESA.

Below, I plot the residuals.

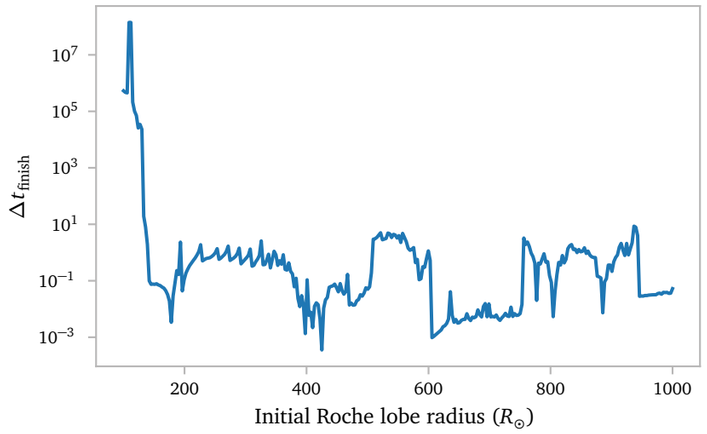


I think it is also interesting to look at the differences between the sophisticated solver using the smaller stellar model and the two methods using the complete stellar model.



The simple integration with the reduced star model agrees well when it is able to survive to the TPAGB branch. Especially for large Roche lobe Radii, the agreement is great, but I think for simplicity and consistency, it is best to use the simple integration with the full model. This way, we use the same integration scheme in both MESA and the `evolve.py` code, which should result in the smoothest transition from `evolve.py` to MESA.

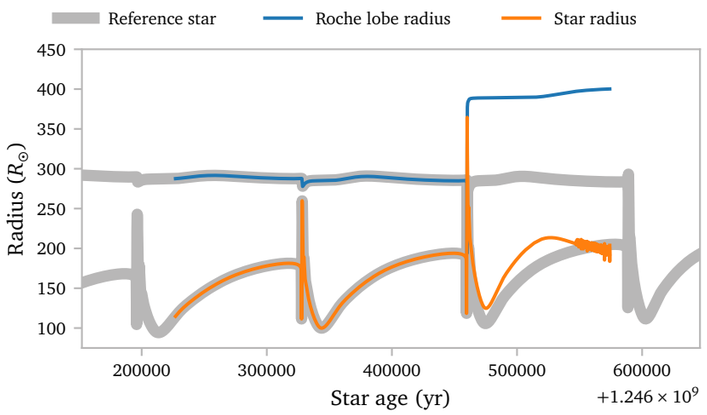
I do however want to make sure the full star with just simple integration reaches the TPAGB phase without much trouble.



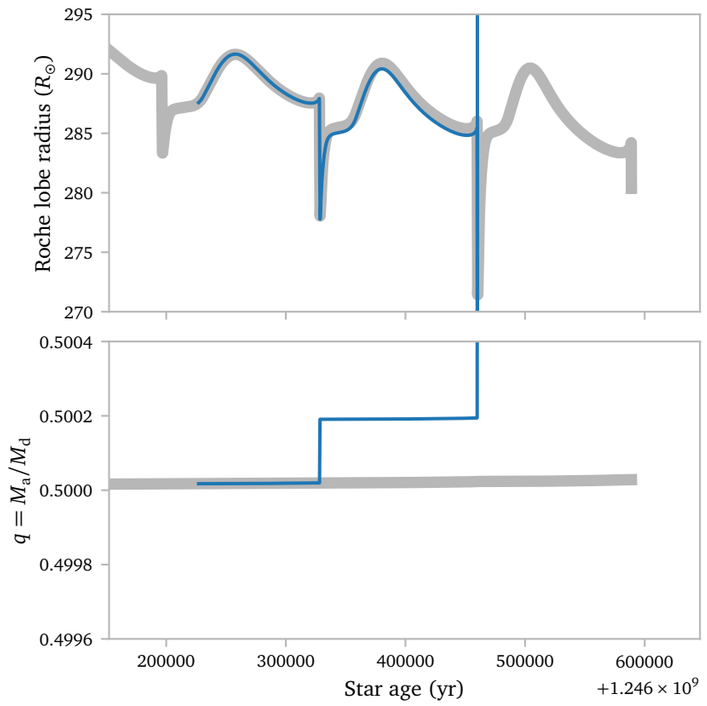
As long as we dont start with radii smaller than about $R_{\text{RL}} \approx 150\,R_\odot$, we *should* be fine.

d.6 Results with Complete Reference Star

When we use the complete stellar model with simple integration only as a starting point, we obtain the following: Below I plot a zoom



The curves follow eachother beautifully!



The Roche lobe radius is slightly smaller after the pulse, due to RLOF mass transfer.

e. Grid

Once I fixed all the problems, I felt like it was *finally* time to simulate multiple star on the cluster in a grid. *However*, the cluster was down when I wanted to transfer all my data and run the grid.

I notified the postmaster and Erik gave the cluster a power cycle and now it is up and running and my models are being simulated at this moment.

f. Analytic Solutions

I have not had time yet

f.1 Predicting Beta Values

g. Improvements

1. For determining a correct starting model, I check the fraction $R_{\rm star}/R_{\rm RL}$ using the radius of star at the point of the radius peak and the initial Roche Lobe radius. Using the peak radius is good, but the Roche lobe radius can vary quite a bit, so it would be better to use the `evolve.py` script to find the Roche lobe radius at the point of the radius peak. This would require some searching but should still be quite easy to implement.
2. The model saving procedure could be updated in a smart way to also work for stars where the maximum radius during a pulse-interpulse cycle is achieved during the interpulse.
3. Change the `MesaGrid` class to use the `Star` object from `evolve.py` instead of the reference history files. This would cut down on loading times since at this point *both* `MesaGrid` as well as `evolve.py` need to load the reference star data, which for the complete model takes a while[3].

[3] A while being approximately 1 minute.

References

Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up*. MNRAS527, 9643.

Saladino, M. I., Pols, O. R., van der Helm, E., Pelupessy, I., & Portegies Zwart, S. (2018) *Gone with the wind: the impact of wind mass transfer on the orbital evolution of AGB binary systems*. A&A618, A50.

Vassiliadis, E. & Wood, P. R. (1993) *Evolution of Low- and Intermediate-Mass Stars to the End of the Asymptotic Giant Branch with Mass Loss*. ApJ413, 641.